# MINI DATABASE DESIGN NOTE

Student Course Registration System

| | |
|---|---|
| **Project type** | Database design / DBMS capstone |
| **Purpose** | Store student details, course information, and course registrations in a structured relational database. |
| **Prepared for** | DBMS course project submission |

## Project summary

This note proposes a small relational database for a college course-registration workflow. It explains the requirements, entities, relationships, tables, keys, constraints, example SQL, and a short testing plan. The design is intentionally compact so it can be implemented in MySQL, PostgreSQL, SQLite, or another relational DBMS.

**Note:** This is a sample project document created from the assignment shown in the screenshot. Replace the cover details and adjust the design to match your own project or instructor's rubric before submitting.

# 1. Problem Statement

Colleges need a consistent way to maintain student records, publish courses, and track which students register for which courses in a semester. Keeping this information in separate spreadsheets can lead to duplicate data, inconsistent course names, and registration errors.

The proposed system stores students and courses once, then records registrations through a linking table. It supports basic tasks such as adding a student, creating a course, registering a student, viewing course rosters, and checking a student's registered courses.

# 2. Objectives and Scope

## Objectives

- Maintain unique student and course records.
- Allow a student to register for multiple courses.
- Allow a course to have multiple registered students.
- Prevent duplicate registration of the same student in the same course offering.
- Use primary and foreign keys to preserve referential integrity.

In scope: student, course, course offering, and registration data. Out of scope: fee payment, attendance, grading, authentication, and timetable conflict resolution.

# 3. Requirements

## Functional requirements

FR1. Add, update, and view student records. FR2. Add and view courses. FR3. Create a course offering for a specific term. FR4. Register a student for an offering. FR5. View the students registered in an offering. FR6. View all offerings registered by a student.

## Non-functional requirements

The database should reduce duplicate data, enforce valid references, use clear table names, and support common queries efficiently. Only authorized application users should be allowed to change records; access control is handled by the application/DBMS configuration.

# 4. Entities and Relationships

| Entity | Purpose | Key attributes |
|---|---|---|
| Student | Stores each student's profile. | student_id (PK), full_name, email, phone |
| Course | Stores reusable course details. | course_id (PK), course_code, title, credits |
| Offering | Represents a course in a particular term/section. | offering_id (PK), course_id (FK), term, section |
| Registration | Connects a student to an offering. | registration_id (PK), student_id (FK), offering_id (FK), registered_on |

## Relationships

- One Course can have many Offerings (1-to-many).
- One Student can have many Registrations (1-to-many).
- One Offering can have many Registrations (1-to-many).
- Student and Offering therefore have a many-to-many relationship resolved by Registration.

# 5. Relational Schema

STUDENT(**student_id**, full_name, email, phone)
COURSE(**course_id**, course_code, title, credits)
OFFERING(**offering_id**, course_id FK → COURSE.course_id, term, section)
REGISTRATION(**registration_id**, student_id FK → STUDENT.student_id, offering_id FK → OFFERING.offering_id, registered_on)

Primary keys uniquely identify rows. Foreign keys ensure that an offering refers to an existing course and a registration refers to an existing student and offering.

## 6. Table Design and Constraints

| Table | Column | Type | Rule |
|---|---|---|---|
| STUDENT | student_id | INTEGER | Primary key; generated ID |
| | full_name | VARCHAR(100) | NOT NULL |
| | email | VARCHAR(150) | NOT NULL; UNIQUE |
| | phone | VARCHAR(20) | Optional |
| COURSE | course_id | INTEGER | Primary key; generated ID |
| | course_code | VARCHAR(20) | NOT NULL; UNIQUE |
| | title | VARCHAR(120) | NOT NULL |
| | credits | INTEGER | CHECK credits > 0 |
| OFFERING | offering_id | INTEGER | Primary key; generated ID |
| | course_id | INTEGER | FK to COURSE; NOT NULL |
| | term, section | VARCHAR | NOT NULL |
| REGISTRATION | registration_id | INTEGER | Primary key; generated ID |
| | student_id | INTEGER | FK to STUDENT; NOT NULL |
| | offering_id | INTEGER | FK to OFFERING; NOT NULL |
| | registered_on | DATE | NOT NULL |

Additional integrity rule: add a UNIQUE constraint on (student_id, offering_id) in REGISTRATION so the same student cannot register for the same offering twice.

## 7. SQL Implementation (Example)

The following SQL uses broadly supported syntax. Auto-generated key syntax differs by DBMS, so adapt ID columns if required by your database.

```
CREATE TABLE Student (
student_id INTEGER PRIMARY KEY,
full_name VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE,
phone VARCHAR(20)
);

CREATE TABLE Course (
course_id INTEGER PRIMARY KEY,
course_code VARCHAR(20) NOT NULL UNIQUE,
title VARCHAR(120) NOT NULL,
credits INTEGER NOT NULL CHECK (credits > 0)
);

CREATE TABLE Offering (
offering_id INTEGER PRIMARY KEY,
course_id INTEGER NOT NULL,
term VARCHAR(20) NOT NULL,
section VARCHAR(10) NOT NULL,
FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

CREATE TABLE Registration (
registration_id INTEGER PRIMARY KEY,
```

```
student_id INTEGER NOT NULL,
offering_id INTEGER NOT NULL,
registered_on DATE NOT NULL,
UNIQUE (student_id, offering_id),
FOREIGN KEY (student_id) REFERENCES Student(student_id),
FOREIGN KEY (offering_id) REFERENCES Offering(offering_id)
);
```

## 8. Sample Queries

```
-- 1. List courses available in a term
SELECT c.course_code, c.title, o.term, o.section
FROM Course c
JOIN Offering o ON o.course_id = c.course_id
WHERE o.term = 'Fall 2026';

-- 2. Show the roster for one offering
SELECT s.student_id, s.full_name, s.email
FROM Registration r
JOIN Student s ON s.student_id = r.student_id
WHERE r.offering_id = 101
ORDER BY s.full_name;

-- 3. Show courses registered by a student
SELECT c.course_code, c.title, o.term, o.section
FROM Registration r
JOIN Offering o ON o.offering_id = r.offering_id
JOIN Course c ON c.course_id = o.course_id
WHERE r.student_id = 1;
```

## 9. Normalization

The design is intended to satisfy Third Normal Form (3NF): student details are stored only in STUDENT, course details only in COURSE, and term/section details only in OFFERING. REGISTRATION contains references to a student and an offering rather than repeating their descriptive details. This reduces update anomalies and unnecessary duplication.

## 10. Testing Plan

| Test case | Expected result |
|---|---|
| Insert a student with a unique email | Student row is saved. |
| Insert another student using the same email | Rejected by UNIQUE constraint. |
| Create an offering for an existing course | Offering row is saved. |
| Create an offering for a non-existent course_id | Rejected by foreign key. |
| Register a student for an offering | Registration row is saved. |
| Register the same student for the same offering again | Rejected by UNIQUE(student_id, offering_id). |
| Run roster query | Only students registered for that offering are returned. |

## 11. Security, Backup, and Limitations

Use database accounts with least-privilege permissions, keep credentials out of source code, validate inputs in the application, and take regular backups. Test restoration periodically. This mini design does not include course capacity limits, waitlists, timetable clashes, grades, or payment records; these can be added in a future version.

## 12. Conclusion

The Student Course Registration System demonstrates a small relational database with clear entities, one-to-many relationships, a many-to-many relationship resolved through a junction table, primary and foreign keys, uniqueness rules, and useful SQL queries. It is a practical starting point for a DBMS capstone and can be expanded as requirements grow.

**Before submission:** Run the SQL in your chosen DBMS, capture evidence of successful table creation and query results if required, and replace sample terms/IDs with data from your own implementation.